Bilkent University
Department of Electrical and Electronics Engineering

Lab Report 05

23 March 2026

EE102 - 01

Muhtasim Hasan Sindeed

22501576

Time-Division Multiplexing and Persistence of Vision Using a Seven-Segment Display

Abstract:

The objective of this laboratory session was to drive four seven-segment displays simultaneously on a Basys 3 FPGA board using a single set of shared cathode control lines. To overcome this hardware constraint, a time-division multiplexing architecture was synthesized using VHDL. A 100 MHz internal clock was divided down to an optimal 1.5 kHz utilizing a 20-bit synchronous counter. The upper bits of this counter acted as a selector for a multiplexer, which rapidly cycled through the active-LOW anodes while routing 16-bit physical switch data to a hexadecimal-to-7-segment decoder. Behavioral simulations confirmed the precise timing of the anode cycling, and hardware implementation successfully demonstrated the "persistence of vision" phenomenon, resulting in a continuous, flicker-free display of four independent hexadecimal digits.

1. Introduction

In digital electronics, minimizing the number of physical connections is crucial for efficient design. The Basys 3 FPGA features four seven-segment displays that share the same seven cathode wires. Consequently, sequential logic and time-division multiplexing

was required to be employed to display distinct values on each digit. By rapidly activating one digit at a time and synchronizing it with the corresponding data input, the hardware exploits the human eye's inability to detect high-frequency flickering, a phenomenon known as persistence of vision. Thus, the objective of this lab was to implement a VHDL-based clock divider, multiplexer, and active-LOW decoder to successfully realize this multiplexed display system.

2. Theory

2.1. Clock Division and Persistence of Vision

The Basys 3 board is driven by an internal oscillator with a frequency of 100 MHz. Because the human eye blends discrete light flashes into a continuous image at frequencies exceeding approximately 40–60 Hz, the 100 MHz clock must be drastically reduced to avoid visual blurring while remaining fast enough to prevent visible flickering.

A digital counter divides the input frequency according to the equation:

$$f_{out} = \frac{f_{clk}}{2^n}$$

Where f_{clk} is the base 100 MHz clock, and n is the specific bit of the counter. By utilizing a 20-bit counter and extracting bits 15 and 16, a 2-bit cyclic sequence is generated at approximately 1.5 kHz, which provides a near ideal refresh rate for the four displays.

2.2. Active-LOW Logic and Hexadecimal Decoding

Both the anodes (digit selectors) and cathodes (segment LEDs) on the Basys 3 operate on active-LOW logic, meaning a logic 0 (0V) completes the circuit and illuminates the LED, while a logic 1 (3.3V) turns it off. The combinational logic decoder must translate a 4-bit binary input (0000 to 1111) into a 7-bit array representing segments a through g.

Table 1: Partial Truth Table for Active-LOW Cathode Decoder (Segments a to g)

Hex Input	Binary Input	Target Display	Cathode Output (a, b, c, d, e, f, g)
0	0000	0	0000001

1	0001	1	1001111
5	0101	5	0100100
A	1010	A	0001000
F	1111	F	0111000

3. Methodology

3.1. Equipments and Components

- Hardware: Digilent Basys 3 Artix-7 FPGA Board (XC7A35TCPG236-1).
- Software: Xilinx Vivado Design Suite.
- I/O Peripherals: 16 physical slide switches (Data Input), 4-digit Seven-Segment Display (Visual Output), 100 MHz internal oscillator (Clock).

3.2. Hardware Architecture

The system was designed in VHDL using three functional blocks:

1. Clock Divider: A synchronous process triggered on the rising edge of the 100 MHz clk signal increments a 20-bit register (divided_clock).
2. Multiplexer (Anode Control): A combinational case statement evaluated the 2-bit activator signal (using bits 15 and 16 of the counter). Depending on the state (00, 01, 10, 11), it drives exactly one anode to 0 and routes the corresponding 4-bit segment of the 16-bit input array to an internal led signal.
3. Cathode Decoder: A Look-Up Table (LUT) evaluates the 4-bit leds signal and outputs the pre-calculated 7-bit active-LOW pattern to the physical a to g pins.

3.3. Synthesis and Physical Mapping

The VHDL entity was mapped to the physical routing of the Basys 3 board utilizing an XDC constraints file. Specific PACKAGE_PIN assignments were made to ensure the 16 switches (V17 through R2) accurately corresponded to the 16-bit input vector, and the a to g cathode pins (W7 through U7) matched the output vector orientation.

4. Results and Analysis

4.1. Behavioral Simulation Verification

Prior to generating the bitstream, a testbench (tb_SevenSeg.vhd) was constructed to simulate the real-time evaluation of the logic. The 16-bit input was stimulated with various hexadecimal representations with state changes occurring every 2.5 ms.

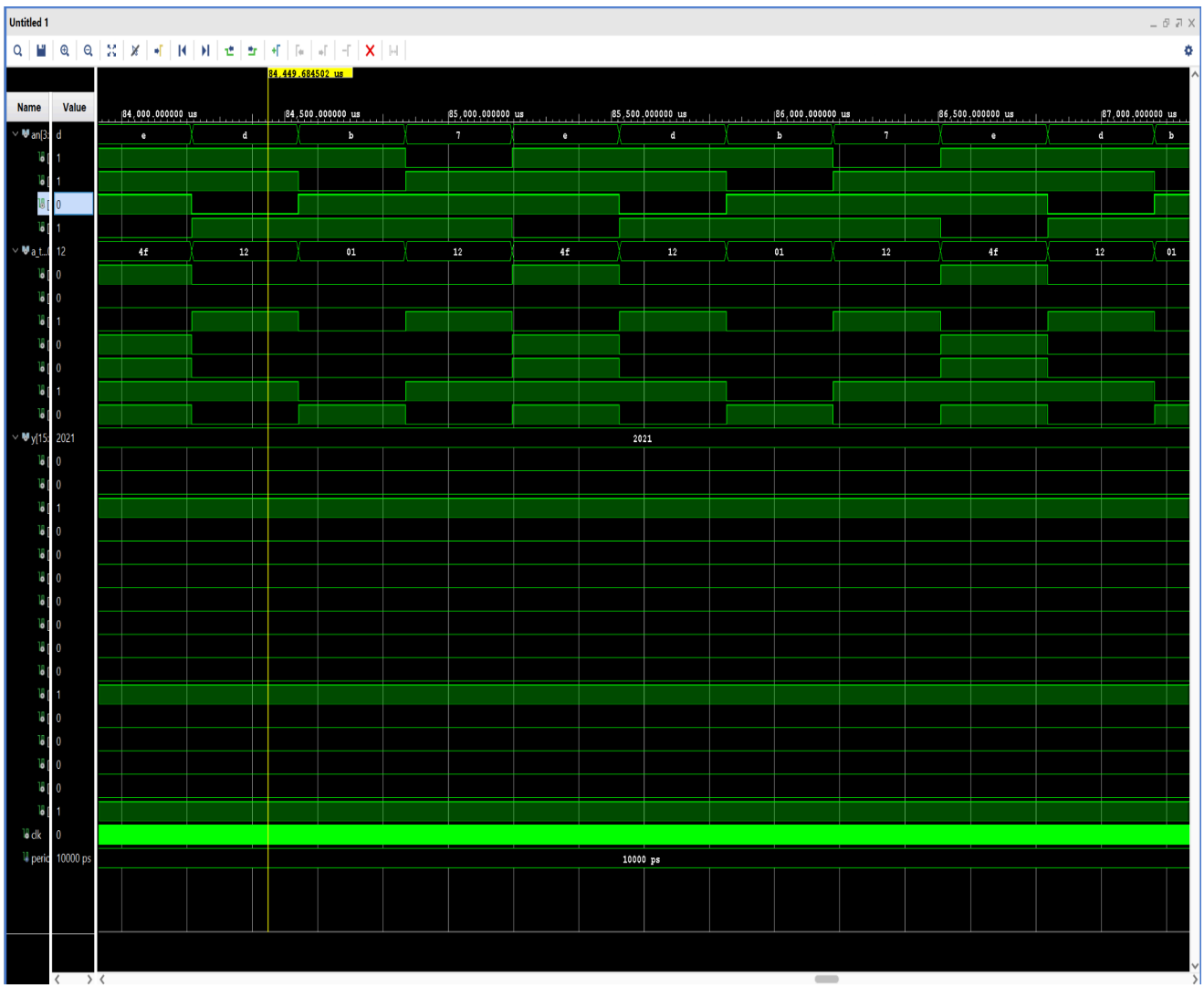

Figure 1: Behavioral simulation waveform verifying the 2-bit selector cycling the active-LOW anode signals and successfully routing the multiplexed data to the cathode decoder.

4.2. Hardware Verification

Upon programming the device, the BASYS 3 board performed exactly as predicted by the behavioral simulation. The 16 physical switches accurately dictated the displayed hexadecimal values (0-9, A-F). No visible flickering was observed, confirming that the 1.5 kHz refresh rate was enough to achieve the persistence of vision effect across all four displays.

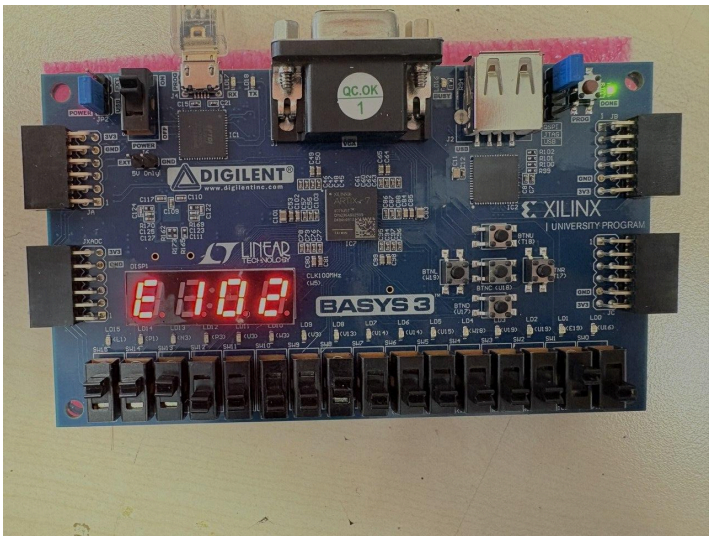

Figure 2: Hardware implementation successfully displaying a 4-digit hexadecimal output.

5. Hardware Considerations

During the synthesis and initial physical testing phases, a critical hardware mapping error required debugging to align the VHDL logic with the physical schematic of the FPGA:

- Cathode Array Bit-Ordering (Inverted Displays): During the first hardware test, specific characters displayed incorrectly; for example, a binary input of 0000 yielded an upside-down "A" rather than a "0". This was diagnosed as a physical mapping mismatch. The VHDL active-LOW array was initially ordered assuming a physical layout of g, f, e, d, c, b, a. Consequently, the output "1000000" intended to turn OFF segment g (the middle bar) actually turned OFF segment a (the top bar). The architectural error was resolved by rewriting the cathode decoder LUT to

strictly follow the a to g left-to-right MSB-to-LSB assignment defined in the .xdc file.

- Refresh Rate Tuning: Initial experimentation with the clock divider revealed that utilizing bits lower than 15 (e.g., bit 10) resulted in "ghosting," where current leakage caused unselected digits to faintly glow. Conversely, utilizing bit 24 resulted in the multiplexer running too slowly, breaking the persistence of vision illusion and causing the digits to visibly flash sequentially. Bits 15 and 16 provided the precise timing balance required by the hardware's specific capacitance.

6. Conclusion

This laboratory session successfully demonstrated the integration of synchronous sequential logic and combinational decoding to overcome physical hardware limitations. The time-division multiplexing architecture accurately routed 16 bits of switch data to a single 7-bit output bus. Through systematic debugging of active-LOW logic and physical pin constraints, a seamless, flicker-free multi-digit display was achieved. The experiment verified both the theoretical mathematics of clock division and the biological phenomenon of persistence of vision in real-time hardware.

Questions to Answer:

1. What is the internal clock frequency of Basys 3?

   The Basys 3 board has a 100 MHz on-board oscillator feeding the FPGA clock input.

2. How can you create a slower clock signal from this one?

   A slower clock can be generated by using a clock divider. The usual lab method is to let a counter run from the 100 MHz clock and then use one of the higher-order counter bits as the slower signal. Each successive counter bit divides the input frequency by 2, so the outputs become progressively slower.

3. Can you create a clock with any arbitrary frequency lower than that of the internal clock? If not, which frequencies can you create?

   In the standard method of clock division, the frequencies we get are only of the form

$$f_{out} = 100MHz/2^n$$

Therefore, the only frequencies possible are of the types 50 MHz, 25 MHz, 12.5MHz etc. We cannot obtain a completely random frequency in this method.

Appendix A: VHDL Source Codes

A.1. Main Module Code (seven_segment.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity seven_segment is
   PORT(
      an: out std_logic_vector(3 downto 0);
      a_to_g: out std_logic_vector(6 downto 0);
      y: in std_logic_vector(15 downto 0);
      clk: in std_logic
   );
end seven_segment;

architecture Behavioral of seven_segment is
   signal divided_clock: std_logic_vector(19 downto 0) := "00000000000000000000";
   signal activator: std_logic_vector(1 downto 0);
   signal leds: std_logic_vector(3 downto 0);

begin


   process(clk)
   begin
      if(rising_edge(clk)) then
         divided_clock <= divided_clock + 1;
      end if;
   end process;
```

```
activator <= divided_clock(16 downto 15);


process(activator, y)
begin
   case activator is
      when "00" =>
         an <= "1110";
         leds <= y(3 downto 0);
      when "01" =>
         an <= "1101";
         leds <= y(7 downto 4);
      when "10" =>
         an <= "1011";
         leds <= y(11 downto 8);
      when "11" =>
         an <= "0111";
         leds <= y(15 downto 12);
      when others =>
         an <= "1111";
   end case;
end process;


process(leds)
begin
   case leds is
      when "0000" => a_to_g <= "0000001"; -- 0
      when "0001" => a_to_g <= "1001111"; -- 1
      when "0010" => a_to_g <= "0010010"; -- 2
      when "0011" => a_to_g <= "0000110"; -- 3
      when "0100" => a_to_g <= "1001100"; -- 4
      when "0101" => a_to_g <= "0100100"; -- 5
      when "0110" => a_to_g <= "0100000"; -- 6
      when "0111" => a_to_g <= "0001111"; -- 7
      when "1000" => a_to_g <= "0000000"; -- 8
```

```
            when "1001" => a_to_g <= "0000100"; -- 9
            when "1010" => a_to_g <= "0001000"; -- A
            when "1011" => a_to_g <= "1100000"; -- b
            when "1100" => a_to_g <= "0110001"; -- C
            when "1101" => a_to_g <= "1000010"; -- d
            when "1110" => a_to_g <= "0110000"; -- E
            when "1111" => a_to_g <= "0111000"; -- F
            when others => a_to_g <= "1111111"; -- Blank
        end case;
    end process;

end Behavioral;
```

A.2. Behavioral Simulation Testbench (tb_SevenSeg.vhd)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Test_Bench is
end Test_Bench;

architecture Behavioral of Test_Bench is
    COMPONENT seven_segment
    Port(
        an: out std_logic_vector(3 downto 0);
        a_to_g: out std_logic_vector(6 downto 0);
        y: in std_logic_vector(15 downto 0);
        clk: in std_logic
    );
    end component;

    signal an: std_logic_vector(3 downto 0);
    signal a_to_g: std_logic_vector(6 downto 0);
    signal y: std_logic_vector(15 downto 0) := (others => '0');
    signal clk: std_logic := '0';

    constant period: time := 10 ns;
```

```
begin
    UUT: seven_segment port map(
        an => an,
        a_to_g => a_to_g,
        y => y,
        clk => clk
    );

    -- 100 MHz Clock Generation
    clock: process
    begin
        wait for period / 2;
        clk <= '1';
        wait for period / 2;
        clk <= '0';
    end process;


    testBench1: process
    begin
        wait for 2.5 ms;
        y <= "0001010001010011";
        wait for 2.5 ms;
        y <= "0001000001110001";
        wait for 2.5 ms;
        y <= "0001100100100011";
        wait for 2.5 ms;
        y <= "0010000000100001";
        wait;
    end process;
end Behavioral;
```

Appendix B: 4-digit Hexadecimal Outputs on the BASYS 3 Board

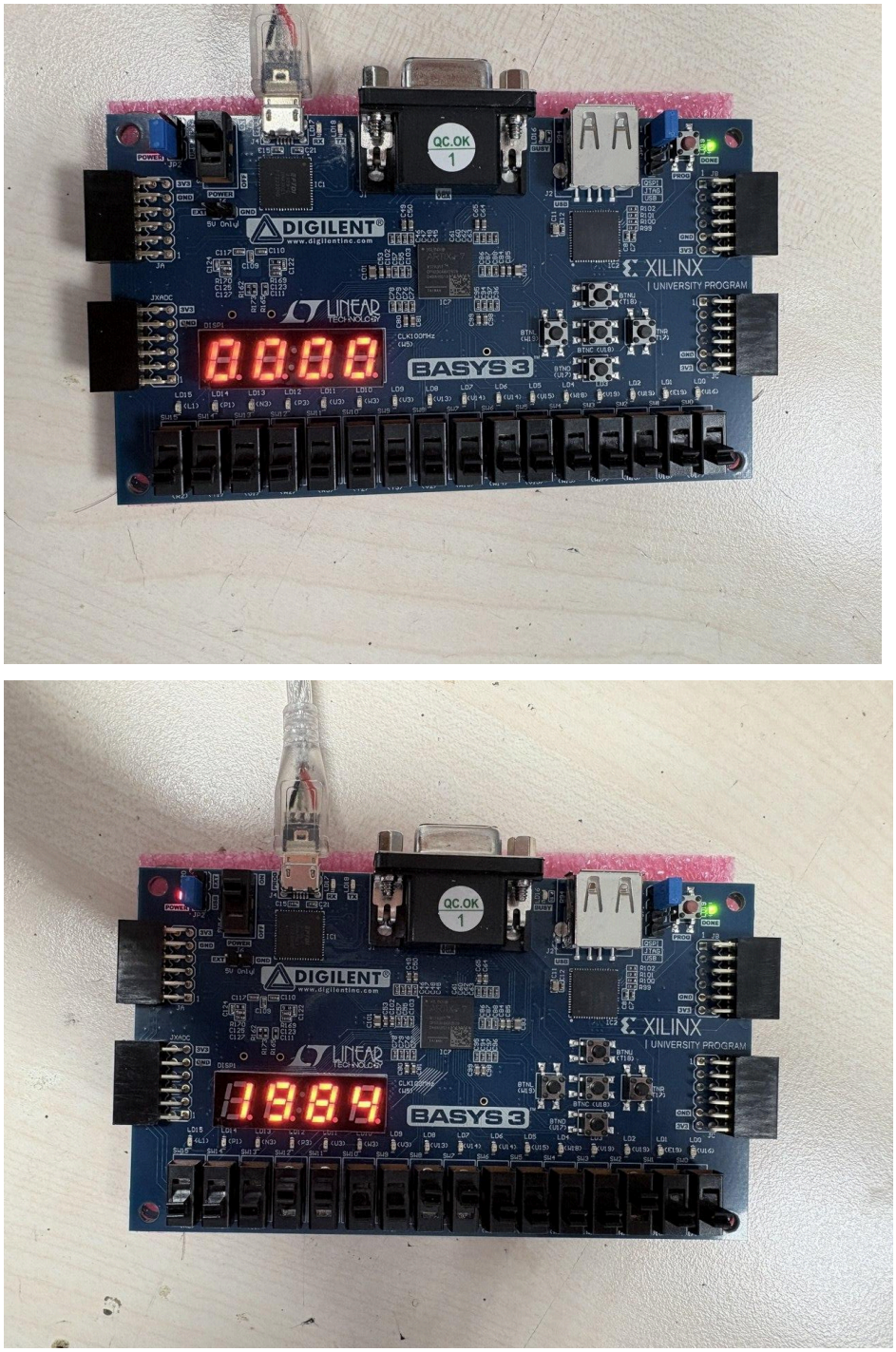

Figure 3: Hardware Implementation of Various 4-digit Hexadecimal Outputs on the BASYS 3 FPGA Board.